

AgentVLN: Towards Agentic Vision-and-Language Navigation *-Supplementary Material-*

Zihao Xin¹, Wentong Li^{1*}(✉), Yixuan Jiang¹, Ziyuan Huang¹, Bin Wang², Piji Li¹, Jianke Zhu³, Jie Qin¹, and Shengjun Huang¹(✉)

¹ Nanjing University of Aeronautics and Astronautics, China

² Shandong University, China

³ Zhejiang University, China

wentong_li@nuaa.edu.cn, huangsj@nuaa.edu.cn

*Project lead ✉Corresponding authors

1 More Discussion on VLN Paradigms

Prevailing VLN methods can be broadly categorized into single-system and dual-system architectures. In this section, we conduct an in-depth comparative analysis from model architecture, error accumulation and correction, training and inference efficiency, and generalization. This analysis clarifies the key limitations of existing paradigms and highlights how AgentVLN is designed to mitigate them. We compare these frameworks in the Fig. 1.

1.1 Single-System Architecture

Single-system frameworks [11, 13, 17, 19, 21, 22] build a monolithic, end-to-end model that maps multimodal observations directly into low-level continuous controls or discrete atomic actions. This paradigm forces a VLM to simultaneously perform high-level reasoning while implicitly learning 3D spatial geometry, kinematics and dynamics, capabilities inherently absent in general-purpose multimodal models. We observe that instruction-tuning compact models ($\leq 7B$) under this objective often severely compromises the model’s core multimodal capabilities. Furthermore, without explicit physical structural priors, these models tend to overfit spurious visual-action correlations, rather than genuinely comprehending 3D spatial topologies. Consequently, under unseen layouts, lighting shifts, or open-ended exploration, these implicit geometric representations often collapse, causing sharp generalization drops.

In continuous long-horizon navigation, even a minute action prediction error at any single step within a single-system framework may cause the subsequent observation state to deviate from the training distribution. During real-world physical deployment, exacerbated by sensor noise or marginal deviations in single-step predictions, the agent inevitably transitions into an out-of-distribution state space s_{t+1} after executing an action at time step t . Lacking an structural understanding of the physical world, subsequent decisions amplify the deviation,

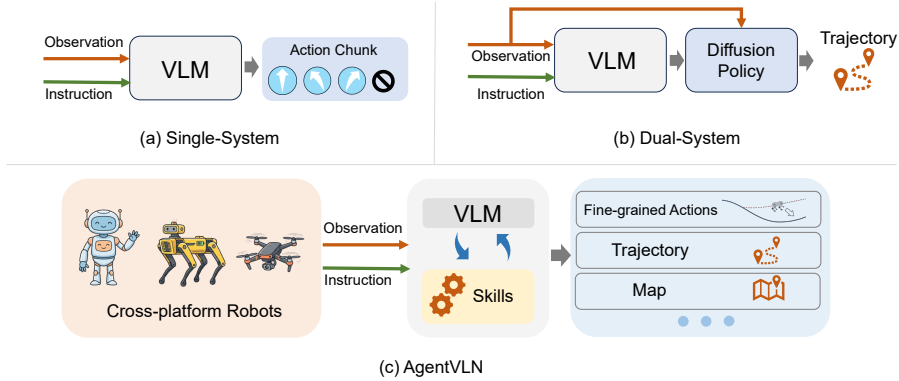


Fig. 1: Comparison of single-system, dual-system and AgentVLN frameworks. Existing single-system approaches directly predict action chunks from observations and instructions, while dual-system approaches couple a VLM with a downstream policy to generate executable trajectories. Our AgentVLN introduces a modular agentic framework, which can support cross-platform robotic systems. By leveraging a plug-and-play skill library, it decouples high-level semantic reasoning from perception and planning, enabling flexible outputs tailored to different tasks. This design provides a new paradigm for next-generation spatially grounded embodied foundation models.

causing errors to accumulate over time and often culminating in catastrophic trajectory failures.

1.2 Dual-System Paradigm

To overcome the black-box nature of single-system approaches, recent research has pivoted toward the dual-system paradigm [6, 10, 14]. This paradigm decouples the system into a low-level motion planner responsible for high-frequency local control (System 1) and a VLM dedicated to low-frequency, high-level decision making (System 2). A typical pipeline involves the VLM predicting target pixels on a 2D image, followed by a diffusion model or alternative generative policy that maps these pixel coordinates into a 3D physical trajectory. However, prior experimental observations suggest that current VLMs with fewer than 30 billion parameters inherently lack the capacity for 3D scale perception within the physical world. When instructions involve complex spatial orientations or intricate scenes, the VLM is highly susceptible to severe scale hallucinations. Consequently, it frequently selects navigation waypoints on the 2D image that are physically unreachable in the 3D space, forcing the subsequent generative policy to blindly fit an invalid path.

In addition, during the training phase, trajectory generation policies not only learn path planning but also passively memorize the idiosyncratic physical attributes of the specific data collection platform. For instance, the model implicitly overfits to the obstacle avoidance safety margins dictated by the robot’s

physical dimensions, as well as the specific kinematics of its chassis and non-holonomic constraints. As a result, the trajectories generated by such policies are exclusively valid for that specific robotic hardware. If the model is deployed on a novel robotic platform, the legacy trajectory generation policy is rendered entirely obsolete, necessitating the massive recollection of hardware-specific trajectory data and extensive retraining from scratch. Ultimately, existing dual-system architectures fundamentally lack plug-and-play flexibility, directly contradicting the foundational objective of developing universally deployable, cross-platform embodied foundation models.

1.3 Our AgentVLN

Distinct from single-system architectures that conflate high-level semantics and low-level kinematics within a unified model, AgentVLN strictly decouples low-level physical control, abstracting it into a highly modular, plug-and-play library of perception and planning skills [12]. Within this framework, the VLM serves purely as a central scheduling “brain”, responsible for comprehending instructions, invoking appropriate skills, and formulating rational navigation plans. This decoupling enables a small VLM backbone ($\leq 3B$) with efficient on-device deployment, and supports zero-shot transfer across robots by swapping only the hardware-specific skill interfaces. When migrating to a new robotic platform, one merely needs to substitute the hardware-specific low-level skill interfaces, completely circumventing the need for computationally expensive model fine-tuning.

Furthermore, to address the spatial perception deficits inherent in existing VLM approaches, AgentVLN refrains from forcing the VLM to implicitly learn or predict 3D depth. Instead, it leverages deterministic perception algorithms to compute feasible physical waypoints, which are subsequently projected onto the 2D image coordinate system as explicit visual prompts. This innovative mechanism elegantly transforms the notoriously complex challenges of 3D spatial understanding and continuous trajectory generation into the 2D semantic comprehension and discrete selection tasks at which VLMs inherently excel. Consequently, this approach fundamentally eradicates the generation of physically invalid navigation waypoints. By simultaneously bypassing the need for cumbersome depth estimation networks and diffusion-based trajectory generation modules, AgentVLN drastically reduces computational overhead, definitively making high-precision autonomous navigation on edge computing platforms a reality.

2 Edge Deployment and Inference Efficiency Analysis

System deployment efficiency and inference latency are critical determinants of the practical utility of VLN frameworks. Burdened by massive parameter counts or a heavy reliance on 3D feature encoding, existing VLN models [6, 7, 10, 11, 13, 17, 18, 22] typically necessitate streaming sensor data via high-bandwidth networks to high-performance cloud server clusters for asynchronous inference.

In stark contrast, AgentVLN achieves real-time inference directly on the Jetson Orin NX edge computing module, providing a practical paradigm for autonomous navigation characterized by low latency, high precision, and exceptional environmental adaptability. Specifically, by leveraging TensorRT and the vLLM framework to implement operator fusion and efficient context management, a single forward pass and autoregressive generation require approximately 3 seconds. This latency is further compressed to a mere 1 second following W4A16 quantization.

In continuous physical navigation, adopting the traditional "stop-think-act" serial logic inevitably subjects the robot to severe latency-induced pauses during the autoregressive generation process. This delay can even lead to catastrophic collisions with dynamic obstacles due to the system's inability to respond in real-time. To definitively resolve this critical bottleneck at the physical execution layer, we innovatively design an Asynchronous Lookahead Inference and Latency Masking mechanism within the system's scheduling framework. We asynchronously decouple the VLM's high-level reasoning thread from the low-level planning and control thread. Driven by planning-layer skills, as the agent navigates toward a local target waypoint, it continuously computes the remaining distance. Upon approaching the destination, it triggers the VLM's subsequent inference cycle in parallel. This lookahead mechanism effectively masks the VLM's computational latency behind the physical transit time of the robot. Consequently, a new navigation waypoint is already fully generated by the time the agent reaches its current destination, thereby facilitating seamless, highly coherent, and high-precision navigation.

3 More Details of Cross-Space Representation Mapping

AgentVLN's cross-space representation mapping method can be broadly applicable to diverse sensors, such as depth cameras and LiDAR. Notably, it pioneers the integration of high-precision geometric priors from LiDAR into the visual decision-making stream of a VLM, thereby realizing high-precision navigation in open environments and scenes subject to severe illumination interference.

Mapping for LiDAR. In large-scale outdoor scenes and complex scenes characterized by drastic illumination variations, LiDAR can acquire depth information with significantly higher accuracy compared to RGB-D depth cameras, which are highly susceptible to ambient light interference and constrained by limited detection ranges. However, the 3D point clouds generated by LiDAR are exceedingly sparse and lack both color and high-dimensional semantic information, whereas VLMs are optimized for dense RGB inputs.

Our mapping bridges this gap by projecting LiDAR-derived 3D waypoints onto the camera image plane as visual prompts. Assuming the extrinsic matrix from the LiDAR coordinate system to the camera coordinate system is denoted as $T_L^C = [R_L^C \mid \mathbf{t}_L^C] \in SE(3)$, and combining this with the global pose $T_W^L =$

$[R_W^L \mid \mathbf{t}_W^L]$ computed by the perception-level skills, a global waypoint $\mathbf{P}_{w,i}$ is first projected into the LiDAR coordinate system:

$$\mathbf{P}_{L,i} = R_W^L \mathbf{P}_{w,i} + \mathbf{t}_W^L. \quad (1)$$

Subsequently, based on T_L^C , the 3D point cloud and the image plane are aligned to calculate the corresponding pixel coordinates $\mathbf{p}_i^{img} = [u_i, v_i, 1]^T$. The intermediate transformation to the camera coordinate system is given by:

$$\mathbf{P}_{C,i} = R_L^C \mathbf{P}_{L,i} + \mathbf{t}_L^C. \quad (2)$$

Because the field of view (FoV) of the LiDAR sensor is significantly broader than that of the camera, a frustum culling mechanism is strictly required to filter out points outside the camera’s view, retaining only the waypoints with a valid depth $Z_{C,i} > 0$ to serve as visual prompts.

During the **local target localization** stage, AgentVLN outputs the target pixel coordinates $\mathbf{p}_{target}^{img} = [u^*, v^*]^T$. Unlike dense RGB-D cameras that permit direct pixel-level depth queries, the LiDAR point cloud becomes extremely sparse upon projection onto the image plane. To address this, we introduce a sparse depth association tracking model. First, the entire LiDAR point cloud of the current frame, $\mathcal{P}_L$, is projected onto the camera image plane using the extrinsic matrix T_L^C and the camera intrinsic matrix K , generating a sparse depth map D_{sparse} . During this projection, the minimum depth value Z_C at each pixel is retained to correctly handle occlusion relationships.

We then employ Inverse Distance Weighting (IDW) to estimate the depth of the target pixel $\hat{d}$:

$$\hat{d} = \frac{\sum_{m=1}^k \omega_m \cdot d_m}{\sum_{m=1}^k \omega_m}, \quad \omega_m = \frac{1}{\|\mathbf{p}_{target}^{img} - \mathbf{p}_m^{img}\|_2}. \quad (3)$$

Here, bounded by a predefined search radius r , a KD-Tree is utilized to rapidly retrieve the k nearest valid LiDAR projection points. Following this depth estimation, the target is re-projected back into the camera coordinate system $\mathbf{P}_{target}^C$ and ultimately into the global world coordinate system $\mathbf{P}_{target}^W$:

$$\mathbf{P}_{target}^C = \hat{d} \cdot K^{-1} \begin{bmatrix} u^* \\ v^* \\ 1 \end{bmatrix}, \quad (4)$$

$$\mathbf{P}_{target}^W = (T_W^L)^{-1} (T_L^C)^{-1} \mathbf{P}_{target}^C. \quad (5)$$

Finally, AgentVLN invokes the planning-level skills to execute precise path planning and navigation control based on these coordinates. Benefiting from this cross-space representation mapping mechanism, the VLM is entirely relieved from the computational burden of processing massive 3D point cloud tensors, while simultaneously inheriting LiDAR’s distinct advantages for outdoor long-horizon navigation and robust resistance to intense light interference.

Mapping for RGB-D. During the **global path navigation** phase, perception-level skills continuously construct an incremental global map in the background and compute the currently traversable waypoints. To enable the VLM to comprehend spatial positional relationships without the burden of incorporating any additional 3D encoders, we generate explicit visual prompts via the cross-space representation mapping mechanism. Suppose at time step t , the agent acquires the observation data $o_t = \{I_t^{rgb}, D_t\}$. The camera intrinsic matrix is defined as

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}.$$

Concurrently, the perception-level skills compute the current global pose $T_t = [R_t \mid \mathbf{t}_t] \in SE(3)$, where $R_t \in SO(3)$ is the rotation matrix, and $\mathbf{t}_t = [x_t, y_t, h_c]^T$ is the translation vector, with h_c denoting the height of the depth camera. Assuming the i -th 3D feasible waypoint output by the perception skill is represented in the world coordinate system as $\mathbf{P}_{w,i} = [X_i, Y_i, 0]^T$, it can be projected into the camera coordinate system $\mathbf{P}_{c,i} = [X_i^c, Y_i^c, Z_i^c]^T$ and subsequently into the pixel coordinate system $\mathbf{p}_i^{img} = [u_i, v_i]^T$ through an inverse transformation:

$$\mathbf{P}_{c,i} = R_t^{-1}(\mathbf{P}_{w,i} - \mathbf{t}_t) = R_t^{-1} \begin{bmatrix} X_i - x_t \\ Y_i - y_t \\ -h_c \end{bmatrix}, \quad (6)$$

$$s \begin{bmatrix} u_i \\ v_i \\ 1 \end{bmatrix} = K \mathbf{P}_{c,i}. \quad (7)$$

Here, $s = Z_i^c$ represents the depth scale factor of this point within the camera coordinate system. Through this cross-space representation mapping mechanism, abstract 3D waypoints are transformed into intuitive visual prompts comprehensible to the VLM. Consequently, the model is only required to make discrete selections and decisions grounded in the high-level semantics of the language instructions.

In the **local target localization** phase, AgentVLN outputs the target pixel coordinates $\mathbf{p}_{target}^{img} = [u^*, v^*]^T$. Initially, it invokes a perception-level skill to extract the depth $d = D_t(u^*, v^*)$ and projects this point back into the local camera coordinate system:

$$\mathbf{P}_{target}^c = d \cdot K^{-1} \mathbf{p}_{target}^{img} = \begin{bmatrix} \frac{(u^* - c_x) \cdot d}{f_x} \\ \frac{(v^* - c_y) \cdot d}{f_y} \\ d \end{bmatrix}. \quad (8)$$

Finally, by integrating the current global pose T_t , the local 3D coordinates are mapped back into the global world coordinate system via a rigid body transformation to obtain the final target coordinates $\mathbf{P}_{target}^w$:

$$\mathbf{P}_{target}^w = R_t \mathbf{P}_{target}^c + \mathbf{t}_t. \quad (9)$$

4 Details of AgentVLN-Instruct Dataset

To bridge the gap in existing datasets regarding the alignment between high-level language instructions and low-level skills, we construct **AgentVLN-Instruct**, a large-scale navigation dataset in the Habitat simulator based on R2R and RxR datasets, covering 61 indoor scenes. Unlike traditional methods that employ a singular control policy throughout the entire navigation process, AgentVLN-Instruct introduces a dynamic stage-routing mechanism conditioned on target visibility. Concretely, at each decision step, the process is routed to either global path navigation or local target localization based on the target’s visibility in I_t^{rgb} . This design mirrors the human wayfinding process, *i.e.*, navigate coarsely first, localize precisely second.

In the **global path planning** phase, the model is required to trigger perception-level skills $\mathcal{F}_{percep}$ and we select the mapped waypoint closest to the expert trajectory as the navigation label. This guides the VLM to learn how to select the most instruction-consistent direction among multiple visual cues. Additionally, random noise is injected to force deviations from the optimal path. When no valid waypoints exist in I_t^{rgb} , the expected output format is switched to fine-grained action sequences, encouraging the model to acquire context-driven self-localization and closed-loop error correction capabilities. In the **local target localization** phase, we design a multi-round, interactive perceptual Chain-of-Thought fine-tuning format that simulate the agent’s active reasoning and perceptual querying behavior, ultimately producing the target pixel coordinates.

Through explicit Chain-of-Thought prompting and lightweight, plug-and-play skill invocation, we directly activate the robust commonsense reasoning and logical discrimination already present in the pre-trained VLM. By bridging 2D visual evidence and 3D physical structure through natural-language-guided perceptual reasoning, our formulation not only improves localization success in complex and ambiguous scenarios but also endows the model’s decision-making process more interpretable, providing a novel paradigm for lightweight general-purpose embodied agents.

5 Additional Implementation Details

We adopt Qwen2.5-VL-3B [2] as the central brain of AgentVLN. To strike an optimal balance between preserving fine-grained visual details and managing token complexity, the agent’s egocentric RGB and depth map observations are uniformly resampled to a resolution of 640×480 . The temporal multimodal historical context length is set to 8 frames. During the instruction-tuning phase, to prevent the catastrophic forgetting of foundational multimodal capabilities and to mitigate overfitting to the simulator’s specific rendering style, we introduce the LLaVA-Video-178K dataset [19] alongside our Habitat-based AgentVLN-Instruct dataset. This strategic co-training effectively activates the large model’s comprehension of universal spatiotemporal dynamics in the real world.

In the AgentVLN training paradigm, the model’s optimization objective is to precisely comprehend the geometric and semantic intents of visual prompts,

and to learn the underlying logic for triggering the corresponding perception or planning skills. This implies that the VLM fundamentally learns the significance of visual cues rather than raw, modality-specific information. We empirically validate this capability during real-world deployment experiments: even when the low-level RGB-D camera is substituted with a LiDAR sensor, as long as the perception-level skill $\mathcal{F}_{percep}$ maps the sparse LiDAR point cloud into a standardized visual prompt, the VLM consistently and accurately interprets the prompt to make correct path selections. This decisively substantiates that AgentVLN is seamlessly compatible with diverse, heterogeneous ranging sensors.

More importantly, AgentVLN exhibits exceptional, zero-shot applicability across diverse hardware platforms. In traditional tightly-coupled models, training data often implicitly embeds the physical characteristics of the specific data collection platform, such as its physical dimensions, chassis kinematics, or turning radius. Conversely, the AgentVLN-Instruct dataset is meticulously curated to exclude any low-level parameters tied to a specific robot’s physical structure. We rigorously decouple all local obstacle avoidance, kinematic constraints, and continuous velocity planning into independent planning-level skills $\mathcal{F}_{plan}$. Consequently, the exact same fine-tuned VLM weights can be directly deployed onto entirely novel robotic platforms without incurring the prohibitive costs of re-training or fine-tuning, thereby realizing a truly universal embodied navigation brain.

6 The Depth Perception Dilemma

Although depth sensors have become inexpensive and ubiquitous in modern robotic hardware, most VLM-based navigation models rarely utilize depth information directly and effectively [3, 11, 19, 22]. The primary reason lies in the inherent modality representation bottlenecks of pre-trained VLMs. Current VLMs [1, 4, 5, 8, 9, 15, 16, 20] are pre-trained on massive large-scale RGB image-text pairs, which yields strong 2D appearance understanding and semantic alignment but provides little capacity to interpret depth maps. As a result, many VLN pipelines are often forced to settle for a suboptimal alternative, relying strictly on first-person RGB images or panoramic views for decision-making. Collapsing inherently 3D navigation into 2D imagery, however, exacerbates spatial scale ambiguity and induces scale hallucinations, making it difficult to reason about distances and spatial relations and often leading to failures in complex scenes.

To compensate for the VLM’s deficit in spatial understanding, recent VLN approaches [7, 10, 17, 21] have attempted to introduce auxiliary depth perception modules. Specifically, they input the RGB images into a cumbersome depth estimation model to extract latent depth features, which are then fused with image features and fed into the VLM. This strategy, however, not only significantly inflates the parameter count and computational overhead but also suffers from a fundamental limitation: depth estimation models typically only output affine-invariant relative depth, rather than absolute depth equipped with a true physical scale. Furthermore, because these models are highly reliant on texture

and illumination priors, they frequently generate erroneous depth maps in complex scenes, inadvertently misleading the VLM into making incorrect navigation decisions. Moreover, such computationally heavy models necessitate deployment on remote cloud servers for inference.

AgentVLN fundamentally resolves the aforementioned dilemmas through its cross-space representation mapping mechanism. Within the AgentVLN framework, the VLM is entirely absolved from the need to comprehend or estimate depth maps, nor does it require any auxiliary depth estimators. Instead, it acquires metric depth information directly from inexpensive depth sensors. This data is subsequently transformed, via the cross-space representation mapping mechanism, into intuitive visual prompts—a representational format at which VLMs naturally excel. This decoupling eliminates scale hallucinations while enabling precise, low-latency navigation on resource-constrained edge platforms.

7 More Qualitative Visualizations

We present qualitative navigation visualizations of AgentVLN across diverse environments in Fig. 2 and Fig. 3. In these scenarios, the agent is required to reach the goal solely based on ego-centric observations while following natural-language instructions. The upper-left corner of each frame displays the current step index, model outputs, and Dist denotes the remaining distance to the goal. To better illustrate the efficiency of our method, we additionally provide a global map visualization on the right side. In this map, the cyan dashed line represents the expert trajectory, while the cyan solid line denotes the actual trajectory executed by the agent. The results demonstrate that the proposed model can consistently interpret instructions and successfully reach the target location across a variety of complex environments. Even when the agent temporarily deviates from the expert trajectory, it can effectively leverage both the instruction semantics and the ego-centric visual context to quickly correct its navigation behavior and recover the correct route. Interestingly, we observe that **AgentVLN occasionally learns to exploit shortcuts during navigation**. When ego-centric observations reveal cues strongly correlated with the target location, the agent may directly move toward the goal, bypassing redundant segments of the instruction-following trajectory. This behavior further highlights the model’s ability to integrate visual grounding with high-level instruction understanding to achieve efficient navigation.

8 Limitations and Future Work

Currently, AgentVLN is trained on AgentVLN-Instruct dataset. Constrained by the prohibitively high costs of physical data collection, this dataset is predominantly built upon the Habitat simulator. However, simulated environments cannot fully capture real-world appearance and dynamics, such as abrupt illumination changes, complex specular reflections, and diverse material properties.



Fig. 2: Qualitative visualization of AgentVLN on long-horizon instruction-following navigation. Each episode shows sampled egocentric RGB observations paired with the corresponding global map. The map overlays the expert trajectory (cyan dashed) and the agent trajectory (cyan solid). The instruction text and step-wise outputs are displayed within each panel. Zoom in for better view.



Fig. 3: Qualitative visualization of AgentVLN on long-horizon instruction-following navigation.

This inherent sim-to-real domain gap inevitably constrains the model’s zero-shot generalization capabilities when deployed in extreme or highly unstructured edge-case scenarios. In our future work, we plan to construct a large-scale, real-world dataset that rigorously aligns the egocentric visual streams of actual physical interactions with corresponding skill-invocation instructions.

Furthermore, empirical observations reveal that compact models remain susceptible to skill invocation hallucinations in complex real-world environments. Moving forward, we will explore a reinforcement learning paradigm grounded in preference optimization. By collecting erroneous trajectories and incorrect skill-triggering instances generated during real-world physical deployment, we plan to construct a comprehensive, large-scale negative sample database, and design a punitive reward function based on these preference alignments to systematically penalize and mitigate the model’s hallucination issues.

Beyond instruction-guided navigation, AgentVLN is designed as a general-purpose embodied agent and can be extended to broader embodied tasks, such as autonomous exploration and semantic mapping. These directions will further evaluate the scalability and cross-task generality of the proposed framework.

References

1. Bai, S., Cai, Y., Chen, R., Chen, K., Chen, X., Cheng, Z., Deng, L., Ding, W., Gao, C., Ge, C., et al.: Qwen3-vl technical report. arXiv preprint arXiv:2511.21631 (2025) [8](#)
2. Bai, S., Chen, K., Liu, X., Wang, J., Ge, W., Song, S., Dang, K., Wang, P., Wang, S., Tang, J., Zhong, H., Zhu, Y., Yang, M., Li, Z., Wan, J., Wang, P., Ding, W., Fu, Z., Xu, Y., Ye, J., Zhang, X., Xie, T., Cheng, Z., Zhang, H., Yang, Z., Xu, H., Lin, J.: Qwen2.5-vl technical report. arXiv preprint arXiv:2502.13923 (2025) [7](#)
3. Cheng, A.C., Ji, Y., Yang, Z., Gongye, Z., Zou, X., Kautz, J., Bıyık, E., Yin, H., Liu, S., Wang, X.: Navila: Legged robot vision-language-action model for navigation. *Robotics: Science and Systems* (2025) [8](#)
4. Li, W., Yuan, Y., Liu, J., Tang, D., Wang, S., Qin, J., Zhu, J., Zhang, L.: Tokenpacker: Efficient visual projector for multimodal llm. *International Journal of Computer Vision* **133**(10), 6794–6812 (2025) [8](#)
5. Liu, H., Li, C., Wu, Q., Lee, Y.J.: Visual instruction tuning (2023) [8](#)
6. Long, Y., Cai, W., Wang, H., Zhan, G., Dong, H.: Instructnav: Zero-shot system for generic instruction navigation in unexplored environment. arXiv preprint arXiv:2406.04882 (2024) [2](#), [3](#)
7. Team, I.: InternVLA-N1: An open dual-system navigation foundation model with learned latent plans (2025) [3](#), [8](#)
8. Wang, P., Bai, S., Tan, S., Wang, S., Fan, Z., Bai, J., Chen, K., Liu, X., Wang, J., Ge, W., et al.: Qwen2-vl: Enhancing vision-language model’s perception of the world at any resolution. arXiv preprint arXiv:2409.12191 (2024) [8](#)
9. Wang, S., Liu, S., Kuang, Y., Wei, X., Liu, Y., Li, Z., Man, Y., Chen, G., Tao, A., Liu, G., et al.: Locateanything: Fast and high-quality vision-language grounding with parallel box decoding. arXiv preprint arXiv:2605.27365 (2026) [8](#)
10. Wei, M., Wan, C., Peng, J., Yu, X., Yang, Y., Feng, D., Cai, W., Zhu, C., Wang, T., Pang, J., Liu, X.: Ground slow, move fast: A dual-system foundation model

- for generalizable vision-and-language navigation. In: International Conference on Learning Representations (2026) [2](#), [3](#), [8](#)
11. Wei, M., Wan, C., Yu, X., Wang, T., Yang, Y., Mao, X., Zhu, C., Cai, W., Wang, H., Chen, Y., et al.: Streamvln: Streaming vision-and-language navigation via slowfast context modeling. arXiv preprint arXiv:2507.05240 (2025) [1](#), [3](#), [8](#)
 12. Xin, Z., Li, W., Jiang, Y., Huang, Z., Wang, B., Li, P., Zhu, J., Qin, J., Huang, S.: Agentvln: Towards agentic vision-and-language navigation. arXiv preprint arXiv:2603.17670 (2026) [3](#)
 13. Xin, Z., Li, W., Jiang, Y., Wang, B., Cong, R., Qin, J., Huang, S.: Decovln: Decoupling observation, reasoning, and correction for vision-and-language navigation. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp. 32410–32420 (2026) [1](#), [3](#)
 14. Xue, X., Hu, J., Luo, M., Shichao, X., Chen, J., Xie, Z., Kuichen, Q., Wei, G., Xu, M., Chu, Z.: Omnnav: A unified framework for prospective exploration and visual-language navigation. arXiv preprint arXiv:2509.25687 (2025) [2](#)
 15. Yuan, Y., Li, W., Liu, J., Tang, D., Luo, X., Qin, C., Zhang, L., Zhu, J.: Osprey: Pixel understanding with visual instruction tuning. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp. 28202–28211 (2024) [8](#)
 16. Yuan, Y., Zhang, H., Li, W., Cheng, Z., Zhang, B., Li, L., Li, X., Zhao, D., Zhang, W., Zhuang, Y., et al.: Videorefer suite: Advancing spatial-temporal object understanding with video llm. In: Proceedings of the Computer Vision and Pattern Recognition Conference. pp. 18970–18980 (2025) [8](#)
 17. Zeng, S., Qi, D., Chang, X., Xiong, F., Xie, S., Wu, X., Liang, S., Xu, M., Wei, X.: Janusvln: Decoupling semantics and spatiality with dual implicit memory for vision-language navigation. arXiv preprint arXiv:2509.22548 (2025) [1](#), [3](#), [8](#)
 18. Zhang, J., Li, A., Qi, Y., Li, M., Liu, J., Wang, S., Liu, H., Zhou, G., Wu, Y., Li, X., et al.: Embodied navigation foundation model. arXiv preprint arXiv:2509.12129 (2025) [3](#)
 19. Zhang, J., Wang, K., Xu, R., Zhou, G., Hong, Y., Fang, X., Wu, Q., Zhang, Z., Wang, H.: Navid: Video-based vlm plans the next step for vision-and-language navigation. Robotics: Science and Systems (2024) [1](#), [7](#), [8](#)
 20. Zhang, Y., Wu, J., Li, W., Li, B., Ma, Z., Liu, Z., Li, C.: Video instruction tuning with synthetic data. arXiv preprint arXiv:2410.02713 (2024) [8](#)
 21. Zheng, D., Huang, S., Li, Y., Wang, L.: Efficient-vln: A training-efficient vision-language navigation model. arXiv preprint arXiv:2512.10310 (2025) [1](#), [8](#)
 22. Zhou, G., Hong, Y., Wu, Q.: Navgpt: Explicit reasoning in vision-and-language navigation with large language models. In: Proceedings of the AAAI Conference on Artificial Intelligence. vol. 38, pp. 7641–7649 (2024) [1](#), [3](#), [8](#)